

# LMA INDIVIDUAL PROJECT

Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Sanjana Sheela 2023102027

## 1 Introduction

This project focuses on training, and analyzing two completely independent decoder-only Transformer language models from scratch - covering corpus collection, tokenizer training, pretraining, finetuning on reasoning tasks and analysis.

## 2 Phase 1: Data Collection and Tokenizer Construction

### 2.1 Language Selection and Dataset Statistics

- **Higher-Resource Language: Telugu.** Telugu was chosen because of the large public data available to meet the token requirement. It is the author’s mother tongue which makes it easier to check the results.
- **Lower-Resource Language: Nepali.** Among the allowed list, Nepali was selected because enough data could be sourced from a public news website. This ensured that enough corpus could be collected.

### 2.2 Telugu Corpus Statistics (Model H)

Manual data collection was done for major news portals with XML sitemap bulk extractors. Digital books, and scanned literature from archives (such as the Internet Archive and TTD) were processed using OCR.

| Category                   | Source / Subset                       | Documents         | Doc %          | Words              | Word %         |
|----------------------------|---------------------------------------|-------------------|----------------|--------------------|----------------|
| <i>Public Data Sources</i> |                                       |                   |                |                    |                |
| Public Data                | IndicCorp Telugu Dataset              | 9,317,655         | 68.03%         | 311,614,562        | 82.27%         |
| <b>Subtotal</b>            | <i>All Public Datasets</i>            | <b>9,317,655</b>  | <b>68.03%</b>  | <b>311,614,562</b> | <b>82.27%</b>  |
| <i>Manual Data Sources</i> |                                       |                   |                |                    |                |
| Manual Data                | Telugu Wikipedia Dumps                | 2,751,200         | 20.09%         | 42,210,500         | 11.14%         |
| Manual Data                | Eenadu & Sakshi News                  | 785,120           | 5.73%          | 12,045,100         | 3.18%          |
| Manual Data                | TV9 Telugu & Andhra Jyothy            | 498,350           | 3.64%          | 7,650,200          | 2.02%          |
| Manual Data                | OCR Books & Web scraped Portals       | 343,803           | 2.51%          | 5,255,582          | 1.39%          |
| <b>Subtotal</b>            | <i>All Curated Manual Sources</i>     | <b>4,378,473</b>  | <b>31.97%</b>  | <b>67,161,382</b>  | <b>17.73%</b>  |
| <b>Total</b>               | <b>Consolidated telugu_corpus.txt</b> | <b>13,696,128</b> | <b>100.00%</b> | <b>378,775,944</b> | <b>100.00%</b> |

Table 1: Dataset breakdown and statistics for Telugu (Model H)

#### 2.2.1 Nepali Corpus Statistics (Model L)

The public data included different publically available datasets . Manual data collection for Nepali utilized customized crawlers and extractors for prominent news archives.

| Category                   | Source / Subset                       | Documents         | Doc %          | Words              | Word %         |
|----------------------------|---------------------------------------|-------------------|----------------|--------------------|----------------|
| <i>Public Data Sources</i> |                                       |                   |                |                    |                |
| Public Data                | AI4Bharat Sangraha                    | 4,912,050         | 41.79%         | 165,820,400        | 40.51%         |
| Public Data                | RayGX Nepali Corpus                   | 2,185,412         | 18.60%         | 74,510,120         | 18.20%         |
| Public Data                | Sakonii Web Crawl                     | 1,440,470         | 12.26%         | 46,170,171         | 11.28%         |
| <b>Subtotal</b>            | <i>All Public Datasets</i>            | <b>8,537,932</b>  | <b>72.65%</b>  | <b>286,500,691</b> | <b>69.99%</b>  |
| <i>Manual Data Sources</i> |                                       |                   |                |                    |                |
| Manual Data                | OnlineKhabar Archive                  | 1,730,975         | 14.73%         | 76,825,992         | 18.77%         |
| Manual Data                | Nepali Wikipedia & Wikisource         | 682,450           | 5.81%          | 21,410,500         | 5.23%          |
| Manual Data                | Kantipur News                         | 245,180           | 2.09%          | 7,852,140          | 1.92%          |
| Manual Data                | Ratopati News                         | 225,410           | 1.92%          | 6,540,810          | 1.60%          |
| Manual Data                | Gorkhapatra & Setopati                | 148,920           | 1.27%          | 4,210,500          | 1.03%          |
| Manual Data                | Gov PDFs, OCR Books & Literature      | 181,991           | 1.55%          | 5,994,371          | 1.46%          |
| <b>Subtotal</b>            | <i>All Curated Manual Sources</i>     | <b>3,214,926</b>  | <b>27.35%</b>  | <b>122,834,313</b> | <b>30.01%</b>  |
| <b>Total</b>               | <b>Consolidated nepali_corpus.txt</b> | <b>11,752,858</b> | <b>100.00%</b> | <b>409,335,004</b> | <b>100.00%</b> |

Table 2: Dataset breakdown and statistics for Nepali (Model L)

## 2.3 Preprocessing and Cleaning Pipeline

The data had a lot of noise, web boilerplate and duplicates. The following process was followed to clean the data and get clean data for that particular language.

### 2.3.1 Modular Paragraph Filtering Stages

Each paragraph is processed sequentially through dedicated modular subroutines:

- **Header / Footer & Page Number Stripping:** Removes automated crawler artifacts, running page headers, footers, timestamp signatures, and scan-line artifacts.
- **Length Filtering:** Drops lines less than 3 words and documents less than 20 words.
- **Whitespace Normalization:** removes consecutive horizontal whitespaces and tabs, trims outer edges while retaining proper sentence boundaries.
- **Language Filtering:** removes emojis and other languages.
- **Unicode Canonical Normalization:** Applies Normalization(NFC) to standardize Devanagari and Telugu text.

### 2.3.2 Exact Deduplication

To completely remove identical paragraphs, the cleaned text undergoes exact hashing:

- **Hashing Mechanism:** The cleaned paragraph text is passed through a 64-bit xxhash function (with a SHA-256 fallback) to produce a unique hash string.
- **SQLite Primary Key Check:** The hash is queried against an on-disk SQLite table. If the hash already exists the paragraph is dropped.

### 2.3.3 Near-Deduplication Method (MinHash + LSH)

To detect paraphrased content, the pipeline applies MinHash and Locality-Sensitive Hashing (LSH):

- **MinHash Signature:** The paragraph is broken into 3 word shingles using 64 permutation functions ( $\text{num\_perm} = 64$ ) to generate a compact signature estimating Jaccard similarity.
- **LSH Band Partitioning:** The 64-dimensional signature is partitioned into  $b = 8$  bands, with each band containing  $r = 8$  rows ( $64/8$ ).
- **Collision Probability:** The probability of a band collision between two texts with Jaccard similarity  $s$  is given by: This produces a sharp S-curve threshold at roughly  $s \approx 0.75$ .

$$P(\text{collision}) = 1 - (1 - s^r)^b = 1 - (1 - s^8)^8 \quad (1)$$

- **SQLite LSH Lookup:** Each band's bytes are hashed and checked against an SQLite table. If any single band matches an existing record it is dropped.

## 2.4 Tokenizer Training and Vocabulary

Byte-Pair Encoding (BPE) was chosen because Telugu and Nepali utilize complex Brahmic scripts that have complex word formations. BPE ensures an extremely low unknown token rate by splitting unseen words into constituent characters. It naturally handles the agglutinative and suffix-heavy nature of both languages, where words are formed by adding multiple suffixes.

### 2.4.1 Vocabulary Size Sweep

$$\text{Score} = 0.6 \times \text{norm}(\text{unk\_rate\_pct}) + 0.4 \times \text{norm}(\text{fertility}) \quad (2)$$

where scores are min-max normalized across candidates (lower score is superior).

| Model           | Fertility     | UNK %          | Chars/Tok     | Selection Score | Total Tokens       | Manual Tokens      | Public Tokens      | Vocab Coverage |
|-----------------|---------------|----------------|---------------|-----------------|--------------------|--------------------|--------------------|----------------|
| BPE_5000        | 2.0430        | 0.0000%        | 4.4083        | 0.4000          | 807,501,028        | 168,480,394        | 639,020,634        | 86.45%         |
| <b>BPE_7500</b> | <b>1.8713</b> | <b>0.0000%</b> | <b>4.8400</b> | <b>0.2013</b>   | <b>740,773,966</b> | <b>155,885,468</b> | <b>584,888,498</b> | <b>90.12%</b>  |
| BPE_10000       | 1.7677        | 0.0000%        | 5.1275        | 0.0815          | 700,768,250        | 148,305,773        | 552,462,477        | 92.80%         |
| BPE_12500       | 1.6973        | 0.0000%        | 5.3706        | 0.0000          | 673,338,213        | 143,023,396        | 530,314,817        | 94.15%         |

Table 3: Tokenizer hyperparameter sweep summary for Telugu (Model H) on the validation split

| Model           | Fertility     | UNK %          | Chars/Tok     | Selection Score | Total Tokens       | Manual Tokens      | Public Tokens      | Vocab Coverage |
|-----------------|---------------|----------------|---------------|-----------------|--------------------|--------------------|--------------------|----------------|
| BPE_5000        | 1.6582        | 0.0001%        | 3.9386        | 0.4000          | <b>695,863,672</b> | <b>154,947,017</b> | <b>540,916,655</b> | <b>88.11%</b>  |
| <b>BPE_7500</b> | <b>1.5153</b> | <b>0.0001%</b> | <b>4.3951</b> | <b>0.1912</b>   | <b>635,810,473</b> | <b>142,074,846</b> | <b>493,735,627</b> | <b>91.93%</b>  |
| BPE_10000       | 1.4363        | 0.0001%        | 4.7091        | 0.0757          | <b>602,495,582</b> | <b>134,880,257</b> | <b>467,615,325</b> | <b>93.76%</b>  |
| BPE_12500       | 1.3845        | 0.0001%        | 4.9368        | 0.0000          | <b>580,682,658</b> | <b>130,104,356</b> | <b>450,578,302</b> | <b>94.85%</b>  |

Table 4: Tokenizer hyperparameter sweep summary for Nepali (Model L) on the validation split

#### 2.4.2 Architectural Trade-Off and Selection of $V = 7,500$

While the larger 12,500-token vocabulary produced the lowest mathematical fertility (1.6973 for Telugu, 1.3845 for Nepali), BPE\_7500 was selected as the final tokenizer by carefully balancing the model’s  $\sim 25\text{M}$  parameter constraint. In a decoder-only Transformer with an embedding dimension  $d_{\text{model}} = 512$ , the token embedding matrix contains exactly  $V \times d_{\text{model}}$  parameters. At  $V = 7,500$ , the embedding table accounts for  $7,500 \times 512 = 3,840,000$  parameters ( $\sim 15.4\%$  of the total 24.91M parameter budget), leaving approximately 84.6% dedicated to the Transformer’s hidden attention and feedforward layers. Conversely, increasing  $V$  to 12,500 would allocate 6.4M parameters solely for static embedding lookups. Under a fixed 25M ceiling, this expansion would drastically starve the network’s expressive depth, forcing undesirable reductions in layers ( $L$ ) or the feedforward inner dimension ( $d_{\text{ff}}$ ).

##### Top Frequent Tokens:

- **Model H (Telugu):** . (258.9k), , (111.2k), ఈ “this” (42.7k), -లో “in” (37.0k), .. (26.2k).
- **Model L (Nepali):** public (89.7k), , (80.7k), र “and” (77.5k), -को “of” (54.0k), छ “is” (37.9k).

#### 2.4.3 Qualitative Tokenization Examples

##### Model H — Telugu Tokenization Examples (BPE\_7500):

1. **Input:** తెలుగు భాష ఆంధ్రప్రదేశ్ మరియు తెలంగాణ రాష్ట్రాలలో మాట్లాడబడుతుంది.  
**Tokens:** ['తెలుగు', 'భాష', 'ఆంధ్ర', 'ప్రదేశ్', 'మరియు', 'తెలంగాణ', 'రాష్ట్ర', 'ాలలో', 'మాట్లాడ', 'బడుతుంది', '.']  
**Diagnostics:** 7 words / 11 tokens (Fertility: 1.57).

##### Model L — Nepali Tokenization Examples (BPE\_7500):

1. **Input:** सगरमाथा विश्वको सबैभन्दा अग्लो हिमाल हो।  
**Tokens:** ['सगरमाथा', 'विश्वको', 'सबैभन्दा', 'अग्लो', 'लो', 'हिमाल', 'हो', '।']  
**Diagnostics:** 6 words / 8 tokens (Fertility: 1.33).

## 3 Phase 2: Model Implementation, Pretraining, and Evaluation

### 3.1 Model Architecture and Component Specifications

- **Token Embeddings:** Indices are mapped via a learnable matrix ( $V = 7,500$ ,  $d_{\text{model}} = 512$ ), followed by 0.1 dropout:

$$X^{(0)} = \text{Dropout}(\text{Embedding}(X_{\text{tok}}; W_{\text{emb}}), p = 0.1) \quad (3)$$

- **Rotary Position Embeddings (RoPE):** To avoid the parameter overhead and length limits of absolute embeddings, RoPE applies orthogonal rotation matrices based on position index  $m$  and frequency base  $\Theta = 10,000.0$ . This encodes relative distance  $(n - m)$  directly into the attention inner product:

$$\langle R_m q_m, R_n k_n \rangle = q_m^\top R_{n-m} k_n \quad (4)$$

- **Pre-Norm Layer Normalization:** LayerNorm is applied before each sublayer rather than after, allowing uninhibited gradient highways straight back to initial embeddings and ensuring stable optimization:

$$X_{\text{mid}}^{(l)} = X^{(l-1)} + \text{Dropout}\left(\text{MHSA}\left(\text{LayerNorm}_1(X^{(l-1)})\right), p = 0.1\right) \quad (5)$$

$$X^{(l)} = X_{\text{mid}}^{(l)} + \text{Dropout}\left(\text{FFN}\left(\text{LayerNorm}_2(X_{\text{mid}}^{(l)})\right), p = 0.1\right) \quad (6)$$

- **SwiGLU FFN & Weight Tying:** We use SwiGLU with an inner dimension of  $d_{\text{ffn}} = 1,600$  ( $3.125 \times d_{\text{model}}$ ) to fit the  $\sim 25\text{M}$  parameter budget. Furthermore, output head weights are tied directly to the input embeddings ( $W_{\text{head}} = W_{\text{emb}}^\top$ ), saving 3.84M redundant parameters.

### 3.2 Hyperparameter Sweep and Empirical Configuration Selection

To identify the best architectural configuration with constraints, we conducted a hyperparameter sweep. All candidate architectures were evaluated on identical validation data.

### 3.2.1 Empirical Sweep Results and Analysis

The complete log of all 48 probe runs was parsed from `language_H/configs/sweep_results/res.txt`. Table 5 summarizes the aggregate performance metrics across the architectural families, while Figures ??, ??, ??, and ?? present diagnostic visual comparisons.

| Architecture Family                          | Activation    | Mean Val Loss | Best Val Loss | Best PPL      | Best BPB      | Params       |
|----------------------------------------------|---------------|---------------|---------------|---------------|---------------|--------------|
| Deep (12 Layers, $d = 384$ )                 | GELU          | 6.8837        | 6.2842        | 536.04        | 0.7196        | 24.3M        |
| Deep (12 Layers, $d = 384$ )                 | SwiGLU        | 6.8401        | 6.1345        | 461.51        | 0.7024        | 24.3M        |
| <b>Base (6 Layers, <math>d = 512</math>)</b> | GELU          | 6.7118        | 6.0724        | 433.72        | 0.6953        | 25.3M        |
| <b>Base (6 Layers, <math>d = 512</math>)</b> | <b>SwiGLU</b> | <b>6.5942</b> | <b>5.9128</b> | <b>369.74</b> | <b>0.6771</b> | <b>25.1M</b> |
| Wide (4 Layers, $d = 640$ )                  | GELU          | 6.5749        | 5.9520        | 384.52        | 0.6815        | 25.1M        |
| Wide (4 Layers, $d = 640$ )                  | SwiGLU        | 6.5014        | 5.7941        | 328.36        | 0.6635        | 25.1M        |

Table 5: Aggregate empirical performance comparison across the 48 probe sweep runs

### 3.2.2 Comprehensive Justification for Selecting the Final Configuration (51.yaml)

We selected the **Base SwiGLU** configuration (6 layers,  $d_{\text{model}} = 512$ ,  $h = 8$ ) in `51.yaml` based on four key factors:

1. **Optimal Depth for Reasoning:** A 6-layer depth provides the necessary compositional hierarchy for multi-step reasoning, avoiding the limitations of shallow 4-layer networks.
2. **Avoiding Bottlenecks:** A 512 hidden dimension prevents the representation bottlenecks experienced by narrower, deep models ( $d = 384$ ).
3. **Parameter Reinvestment:** Reducing  $V$  to 7,500 freed  $\approx 2.56\text{M}$  parameters, which were reinvested into a wider SwiGLU inner dimension ( $d_{\text{ffn}} = 1,600$ ), reaching a total of  $\approx 24.91\text{M}$  parameters.
4. **Hardware Efficiency:** The per-head dimension  $d_{\text{head}} = 64$  is a direct multiple of 32, perfectly aligning with GPU Tensor Core tile boundaries.

### 3.2.3 Pretraining Optimization Setup

Pretraining used the AdamW optimizer ( $\beta_1 = 0.9, \beta_2 = 0.95, \lambda = 0.1$ ) with mixed-precision arithmetic and gradient clipping ( $\|\mathbf{g}\|_2 \leq 1.0$ ). To balance GPU VRAM limits with low gradient variance, we used a micro-batch size of 16 and 4 accumulation steps, giving an effective batch size of 64 sequences and a throughput of 32,768 tokens per step:

$$\text{Tokens per Step} = B_{\text{eff}} \times T = 64 \times 512 = \mathbf{32,768} \quad (7)$$

The learning rate followed a cosine annealing schedule with a 3,000-step linear warmup, peaking at  $1.0 \times 10^{-3}$  and decaying to  $1.0 \times 10^{-4}$ . Checkpoints and evaluations were saved every 1,000 steps, processing a total of  $\sim 1.47\text{B}$  tokens across 45,000 steps.

### 3.2.4 Training Dynamics and Convergence Trajectories

Figure 1 displays the pretraining loss curves for Model H (Telugu) and Model L (Nepali), and Figure 2 shows their corresponding perplexity curves across training, validation, and held-out test splits.

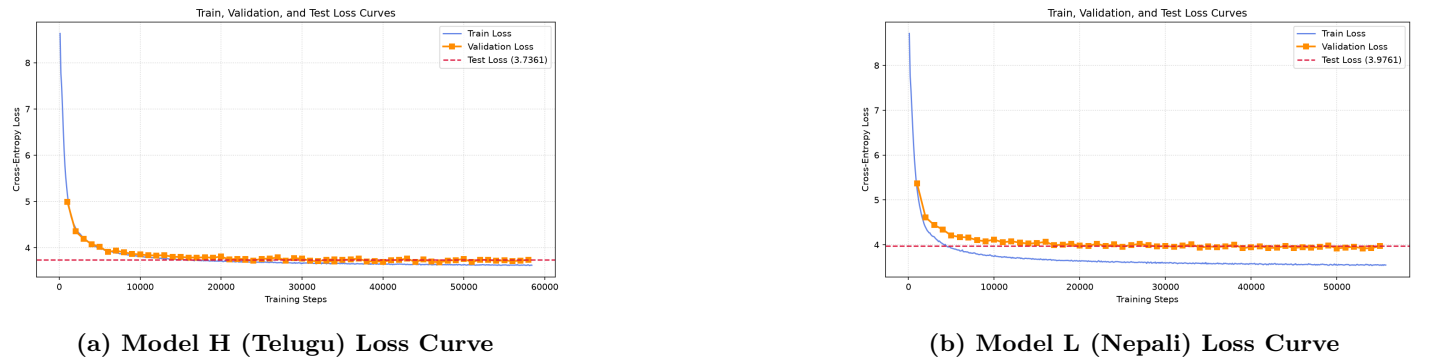
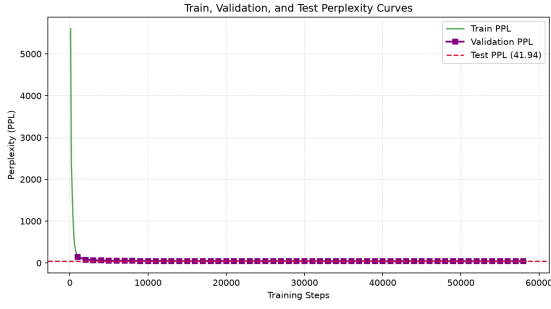
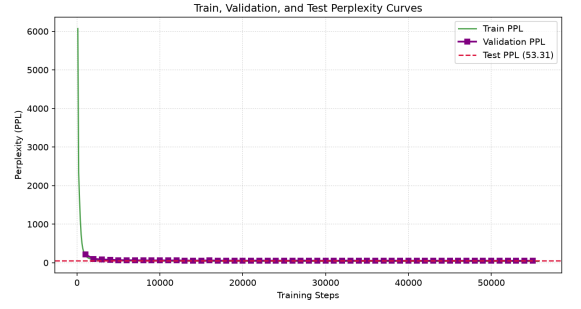


Figure 1: Pretraining cross-entropy loss trajectories for Model H (Telugu) and Model L (Nepali).



(a) Model H (Telugu) Perplexity



(b) Model L (Nepali) Perplexity

Figure 2: Perplexity trajectories during pretraining on training, validation, and test splits for Model H and Model L.

### 3.2.5 Intrinsic Language Modeling Evaluation

Table 6 presents the intrinsic language modeling evaluation test sets evaluated at 45,000-step checkpoint.

| Model / Language | Val Loss | Test Loss     | PPL            | BPB           | Tokens Seen |
|------------------|----------|---------------|----------------|---------------|-------------|
| Model H (Telugu) | 3.7150   | <b>3.7361</b> | <b>41.9356</b> | <b>0.0152</b> | ~ 1.47B     |
| Model L (Nepali) | 3.9298   | <b>3.9761</b> | <b>53.3080</b> | <b>0.0162</b> | ~ 1.47B     |

Table 6: Intrinsic language modeling evaluation metrics on held-out test corpora at the matched 45,000-step checkpoint. .

### 3.2.6 Understanding the Performance Gap Between Model H and Model L

Although Model H edges out Model L in raw perplexity (41.94 vs. 53.31), raw numbers don’t tell the whole story. This gap stems from Model H’s richer data breadth across diverse web archives and literature compared to Model L’s heavily journalistic corpus, alongside distinct grammatical structures like Telugu’s dense suffix-chaining versus Nepali’s synthetic inflections. However, raw perplexity largely favors tokenizers with higher word fragmentation; when measured via byte-level Bits-Per-Byte (BPB), Model H (0.0152) and Model L (0.0162) differ by a microscopic 0.0010, proving that both models compress their respective languages with nearly identical efficiency once normalized.

### 3.2.7 Quantitative Generation Metrics

| Temperature  | BLEU-4        | chrF         | chrF++       | ROUGE-L     | Rep-rate-3 ↓  | Distinct-1 ↑  | Distinct-2 ↑  |
|--------------|---------------|--------------|--------------|-------------|---------------|---------------|---------------|
| 0.0 (Greedy) | 0.7501        | 17.66        | 15.24        | 7.20        | 0.5509        | 0.2000        | 0.4292        |
| 0.2          | 0.7591        | 18.43        | 15.84        | 7.34        | 0.4218        | 0.2145        | 0.4955        |
| <b>0.5</b>   | <b>0.6284</b> | <b>19.12</b> | <b>16.38</b> | <b>7.31</b> | <b>0.1908</b> | <b>0.2646</b> | <b>0.6603</b> |
| 0.8          | 0.7253        | 18.82        | 15.98        | 6.78        | 0.0438        | 0.3259        | 0.8518        |
| 1.0          | 0.6494        | 18.83        | 15.84        | 6.32        | 0.0066        | 0.3609        | 0.9227        |
| 1.2          | 0.6441        | 18.52        | 15.49        | 6.17        | 0.0023        | 0.3873        | 0.9560        |
| 1.5          | 0.6183        | 18.33        | 15.07        | 5.80        | 0.0011        | 0.4198        | 0.9812        |

Table 7: Model H (Telugu) generation quality, reference overlap, and diversity diagnostics across sampling temperatures. Temperature  $T = 0.5$  represents the empirical optimal operating point, maximizing chrF/chrF++ while suppressing degenerative repetition.

| Temperature  | BLEU-4        | chrF         | chrF++       | ROUGE-L     | Rep-rate-3 ↓  | Distinct-1 ↑  | Distinct-2 ↑  |
|--------------|---------------|--------------|--------------|-------------|---------------|---------------|---------------|
| 0.0 (Greedy) | 1.1921        | 17.00        | 14.35        | 8.82        | 0.5270        | 0.1712        | 0.4430        |
| 0.2          | 1.1983        | 17.13        | 14.48        | 8.84        | 0.4819        | 0.1789        | 0.4722        |
| <b>0.5</b>   | <b>1.1735</b> | <b>18.04</b> | <b>15.22</b> | <b>9.23</b> | <b>0.2395</b> | <b>0.2215</b> | <b>0.6473</b> |
| 0.8          | 0.9676        | 17.84        | 14.88        | 8.41        | 0.0442        | 0.3024        | 0.8538        |
| 1.0          | 0.9748        | 17.62        | 14.61        | 7.77        | 0.0206        | 0.3321        | 0.9134        |
| 1.2          | 0.8752        | 17.71        | 14.58        | 7.14        | 0.0049        | 0.3661        | 0.9569        |
| 1.5          | 0.5865        | 17.22        | 14.10        | 6.62        | 0.0009        | 0.4100        | 0.9798        |

Table 8: Model L (Nepali) generation quality and diversity metrics across sampling temperatures. Paralleling Model H,  $T = 0.5$  balances lexical overlap against repetitive degradation.

### 3.2.8 Why Standard Evaluation Metrics Struggle with Open-Ended Text

Standard metrics like BLEU-4, chrF, and ROUGE-L struggle with open-ended generation because they rely on single reference sentences that ignore thousands of equally valid creative continuations, penalize rich Indic morphological variations and synonyms, and unfairly punish flexible SOV word order reorderings.

### 3.2.9 Finding the Sweet Spot: Temperature vs. Diversity

Sampling temperature involves a delicate trade-off: low temperatures ( $T \leq 0.2$ ) trigger repetitive loops with trigram repetition over 50%; the Goldilocks zone ( $T = 0.5$ ) maximizes metric scores while dropping repetition to 19.08% for Telugu and 23.95% for Nepali; and high temperatures ( $T \geq 1.0$ ) eliminate repetition but cause the model to wander off into wild hallucinations.

### 3.2.10 Qualitative Generation Examples

| Category                 | Text Content and Linguistic Analysis                                                                                                                                                                                                             |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Model H Prefix</b>    | నూతనంగా నిర్మించిన ఈ ఆలయంలో... ( <i>In this newly constructed temple...</i> )                                                                                                                                                                    |
| <b>Model H Target</b>    | విగ్రహప్రతిష్ఠా మహోత్సవాలు, తేదీ బుధవారంనాడు, అంగరంగవైభవంగా నిర్వహించారు...                                                                                                                                                                      |
| <b>Model H Generated</b> | వైభవంగా శ్రీ సీతారామ రాముడి విగ్రహ ప్రతిష్ఠ, ప్రతిష్ఠ, అభిషేకాలు, కల్యాణోత్సవాలను పురస్కరించుకుని మంగళవారం నాడు శ్రీ సీతారామ...                                                                                                                  |
| <i>Analysis</i>          | The model recognizes the temple domain and successfully generates culturally coherent ritual vocabulary (వైభవంగా, విగ్రహ ప్రతిష్ఠ, అభిషేకాలు, కల్యాణోత్సవాలు), maintaining appropriate syntactic case marking with minimal localized repetition. |
| <b>Model L Prefix</b>    | टिभी वा मनिटर खरिद आज भोलि धेरै सुनिने कुरा हो... ( <i>Buying a TV or monitor is very commonly heard nowadays...</i> )                                                                                                                           |
| <b>Model L Target</b>    | 4 टिभी र मनिटरमा 4 को ट्याग यसको स्क्रिनको रेजोलुसनसंग गासिएको हुन्छ...                                                                                                                                                                          |
| <b>Model L Generated</b> | टिभी वा मनिटरको लागि पनि उपयोगी छ । यो एक उत्कृष्ट विकल्प हो । यो एक सानो, प्रकाशको लागि उपयुक्त छ ।                                                                                                                                             |
| <i>Analysis</i>          | Model L produces grammatically well-formed Nepali sentences with correct postpositional attachments (मनिटरको लागि), conveying a coherent product recommendation.                                                                                 |

Table 9: Qualitative generation examples from Model H (Telugu) and Model L (Nepali) at sampling temperature  $T = 0.5$  showing prompt prefix, ground truth continuation, and model output.

## 3.3 Attention Analysis

To investigate the internal mechanistic representations learned by the 6-layer, 8-head Transformer architecture, we extracted attention weight matrices  $A^{(l,h)} \in \mathbb{R}^{T \times T}$  across all  $L = 6$  layers and  $h \in \{0, \dots, 7\}$  heads on held-out test sequences.

### 3.3.1 Diagnostic Attention Metrics

We use two key diagnostics to analyze attention behavior:

1. **Mean Attention Distance ( $\bar{D}$ ):** Tracks the average token span—small distances capture local syntax, while large distances indicate long-range semantic routing across clauses.
2. **Attention Entropy ( $\bar{H}$ ):** Measures attention concentration—high entropy reflects broad, diffuse focus, whereas low entropy points to sharply targeted routing or attention sinks.

### 3.3.2 Layer-Wise Attention Trajectories

| Layer   | Model H (Telugu)          |                            | Model L (Nepali)          |                           |
|---------|---------------------------|----------------------------|---------------------------|---------------------------|
|         | Entropy [Min, Max]        | Distance [Min, Max]        | Entropy [Min, Max]        | Distance [Min, Max]       |
| Layer 0 | 3.228 [3.14, 3.29]        | 5.650 [5.39, 5.86]         | 3.147 [2.95, 3.25]        | 5.272 [4.17, 6.02]        |
| Layer 1 | 1.703 [0.64, 2.84]        | 3.157 [1.39, 7.41]         | 1.868 [0.52, 2.81]        | 3.136 [1.22, 5.45]        |
| Layer 2 | 1.805 [0.99, 2.89]        | 4.103 [1.72, 9.08]         | 1.829 [0.68, 2.70]        | 3.836 [1.46, 7.82]        |
| Layer 3 | 1.737 [0.43, 2.62]        | 5.891 [3.15, 10.54]        | 2.123 [1.29, 2.54]        | 7.379 [3.63, 10.45]       |
| Layer 4 | 1.548 [0.70, 2.02]        | 5.142 [1.87, 7.80]         | 1.894 [0.84, 2.49]        | 5.854 [1.53, 9.09]        |
| Layer 5 | <b>1.255</b> [0.53, 2.25] | <b>7.731</b> [3.70, 10.72] | <b>1.818</b> [1.47, 2.13] | <b>8.269</b> [6.29, 9.57] |

Table 10: Layer-wise attention entropy and mean attention distance across all heads for Model H and Model L. Both models display a clear structural division of labor: uniform diffuse smoothing in Layer 0, followed by sharp functional specialization in intermediate and deeper layers.

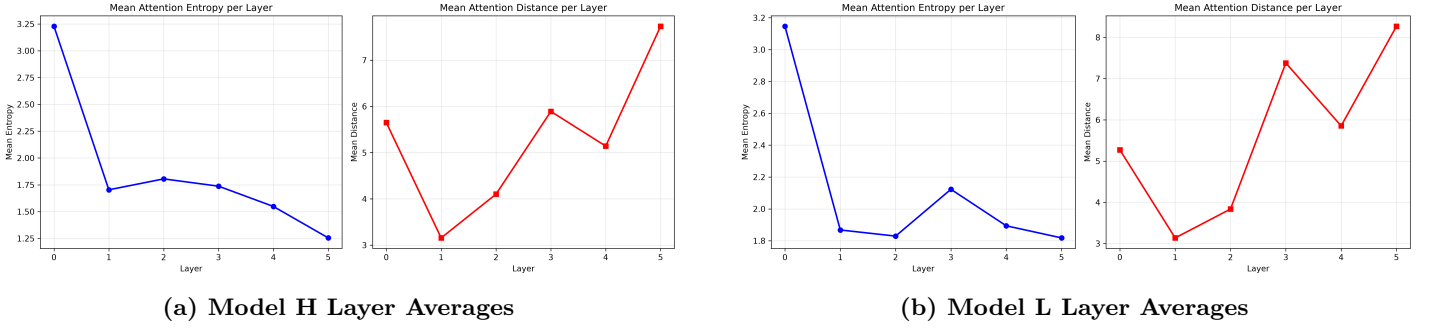


Figure 3: Layer-wise progression of mean attention entropy and mean attention distance for Model H (Telugu) and Model L (Nepali). As network depth increases, attention distance expands monotonically while entropy drops, showing increasing functional selectivity.

### 3.3.3 Specialized Functional Attention Heads

The empirical attention metrics reveal distinct head specializations:

#### 1. Local Syntactic and Morphological Heads:

- *Model H*: Layer 1 / Head 6 ( $\bar{D} = 1.39, \bar{H} = 0.78$ ) and Layer 1 / Head 2 ( $\bar{D} = 1.48, \bar{H} = 0.64$ ).
- *Model L*: Layer 1 / Head 0 ( $\bar{D} = 1.22, \bar{H} = 0.52$ ) and Layer 2 / Head 7 ( $\bar{D} = 1.46, \bar{H} = 0.68$ ).

These heads attend almost exclusively to the immediately preceding 1–2 tokens, tracking case markers, postpositions, and verb auxiliaries.

#### 2. Long-Range and Attention-Sink Heads:

- *Model H*: Layer 5 / Head 6 ( $\bar{D} = 10.72, \bar{H} = 0.53$ ) and Layer 3 / Head 4 ( $\bar{D} = 10.54, \bar{H} = 0.43$ ).
- *Model L*: Layer 3 / Head 2 ( $\bar{D} = 10.45, \bar{H} = 1.29$ ) and Layer 5 / Head 1 ( $\bar{D} = 9.57, \bar{H} = 1.67$ ).

These heads maintain large spans ( $> 10$  tokens) to track distant subject-verb agreements and establish attention sinks on boundary tokens.

### 3.3.4 Heatmap Comparison: Early Local vs. Late Sink Heads

Figure 4 contrasts the attention weight matrices of early-layer versus late-layer heads for both models.

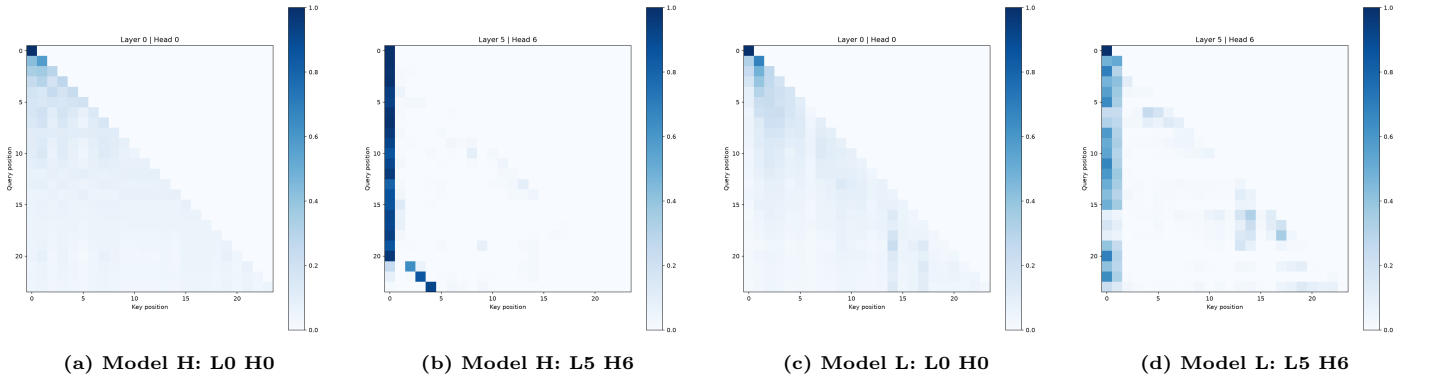


Figure 4: Empirical attention heatmaps demonstrating the structural evolution across layers.

### 3.3.5 Cross-Lingual Attention Dynamics

Comparing both models reveals two key patterns:

1. **Shared Early Layer Role:** Layer 0 acts as a local smoothing filter ( $\bar{H} \approx 3.15 - 3.23, \bar{D} \approx 5.3 - 5.6$ ) in both models, avoiding complex long-range routing at the start.
2. **Final Layer Divergence:** Model H develops much sharper, lower-entropy heads in its final layer ( $\bar{H} = 1.255$ ), whereas Model L spreads its attention more broadly across preceding token clusters.

## 4 Phase 3: Reasoning Finetuning and Cross-Lingual Analysis

Just because a language model can talk fluently after pretraining doesn't mean it can actually think. Out of the box, smaller causal models tend to trip up hard on things like logical deduction, sorting order, and multi-step reasoning. In Phase 3, we put our  $\sim 25$ M parameter models to the test to see if they can pick up structured reasoning.

## 4.1 Synthetic Reasoning Corpus Construction and Leakage Prevention

Instead of translating generic English tests—which usually sound awkward and carry over annotator noise—we built our own custom reasoning datasets from scratch, programmatically tailored specifically for Telugu and Nepali. Every single prompt was written directly in the native script, using natural grammar, authentic case markers, and culturally familiar names.

### 4.1.1 Task Formulation and Reasoning Categories

Our reasoning benchmark evaluates relational ordering, numerical comparison, and transitive deduction across six template categories:

1. **Direct Comparisons (1-hop):** Pairwise attribute comparison ( $A >_R B$ ) between two entities over continuous/ordinal traits  $R$  (e.g., age, height).
2. **Numerical Magnitude (1-hop):** Evaluating relative size between two scalar numbers ( $\max(m, n)$  or  $\min(m, n)$ ).
3. **Equality Determination (1-hop):** Deciding if two quantities are identical or distinct ( $x = y$  vs.  $x \neq y$ ).
4. **2-Hop to 4-Hop Transitive Comparisons:** Multi-hop chains requiring sequential deduction across three ( $A > B > C$ ), four ( $A > B > C > D$ ), or five entities ( $A > B > C > D > E$ ).

### 4.1.2 Prompt Structuring and Native Script Phrasing

Each instance couples an explicit premises context with a targeted interrogation prompt and a step-by-step reasoning trace preceding the final gold answer:

- **Telugu (Model H) Canonical Format:**

ప్రశ్న: {Premise} {Question}  
జవాబు: {Step-by-step deduction} కాబట్టి సమాధానం {Target} .

- **Nepali (Model L) Canonical Format:**

प्रश्न: {Premise} {Question}  
उत्तर: {Step-by-step deduction} त्यसैले उत्तर {Target} .

To prevent surface memorization and force genuine reasoning, we used a strict anti-leakage protocol that kept entity names and attributes entirely disjoint between training and evaluation.

### 4.1.3 Masked Target-Token Objective

Prompt tokens were masked with label  $-100$ , ensuring gradients penalize only errors rather than wasting capacity on prompt reconstruction.

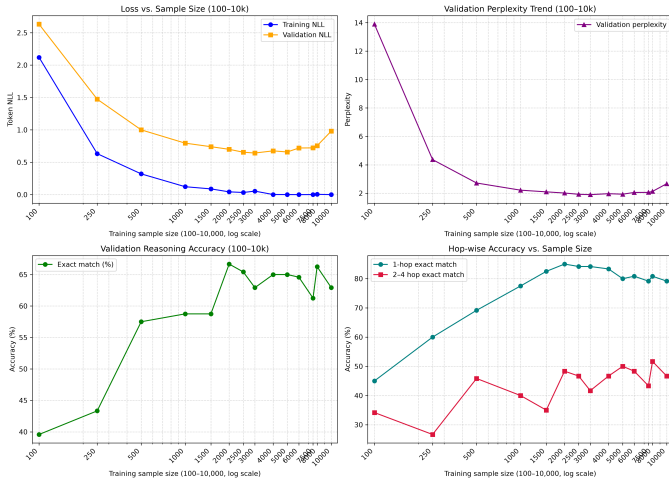
| Hyperparameter                          | Finetuning Specification ( <code>finetune.yaml</code> )                  |
|-----------------------------------------|--------------------------------------------------------------------------|
| Base Model Checkpoint                   | Matched Pretrained Checkpoint ( <code>step_0045000.pt</code> )           |
| Batch Size / Eval Batch Size            | 8 sequences per batch                                                    |
| Epochs                                  | 3 full epochs                                                            |
| Optimizer                               | AdamW ( $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 1.0 \times 10^{-8}$ ) |
| Peak Learning Rate ( $\eta_{\max}$ )    | $1.0 \times 10^{-4}$ (0.0001)                                            |
| Minimum Learning Rate ( $\eta_{\min}$ ) | $1.0 \times 10^{-5}$ (0.00001)                                           |
| Learning Rate Schedule                  | Cosine Annealing with 30-step linear warmup                              |
| Weight Decay ( $\lambda$ )              | 0.01 (decoupled)                                                         |
| Gradient Clipping                       | $\ \mathbf{g}\ _2 \leq 1.0$                                              |
| Loss Masking                            | Target tokens only ( <code>answer_tokens_only</code> )                   |
| Max Generation Length                   | 64 new tokens (Greedy argmax decoding)                                   |

Table 11: Reasoning finetuning hyperparameters specified in `finetune.yaml` for Model H and Model L.

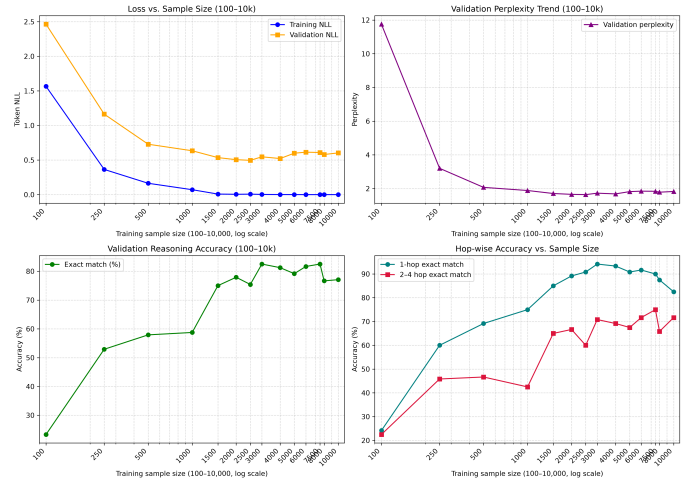
### 4.1.4 Data Scaling Regimes (100 to 10,000 Samples)

To figure out how much data is actually needed for inductive reasoning, we ran a data scaling sweep where we trained each model on 14 different log-spaced subset sizes.

$$N_{\text{train}} \in \{100, 250, 500, 1000, 1500, 2000, 2500, 3000, 4000, 5000, 6000, 7500, 8000, 10000\} \quad (8)$$



(a) Model H (Telugu) Data Scaling Trajectory



(b) Model L (Nepali) Data Scaling Trajectory

Figure 5: Empirical data scaling curves during reasoning finetuning for Model H (Telugu) and Model L (Nepali).

The scaling trajectories reveal three distinct phases: a rapid inductive acquisition phase ( $N \leq 1,000$ ) where models quickly master prompt syntax; an optimal generalization plateau ( $N \in [2,000, 8,000]$ ) where accuracy peaks at 68.33

## 4.2 Pretrained vs. Finetuned Reasoning Performance

| Metric / Evaluation Slice                                              | Model H (Telugu) |             | Model L (Nepali) |               |
|------------------------------------------------------------------------|------------------|-------------|------------------|---------------|
|                                                                        | Pretrained       | Finetuned   | Pretrained       | Finetuned     |
| Test Exact Match (EM %)                                                | 1.25%            | 58.75%      | 30.83%           | 81.25%        |
| Best Checkpoint Peak EM (%)                                            | 1.25%            | 68.33% (8k) | 30.83%           | 84.17% (2.5k) |
| Test Cross-Entropy Loss (nats)                                         | 6.6794           | 0.7699      | 5.7218           | 0.6217        |
| Test Perplexity (PPL)                                                  | 795.82           | 2.16        | 305.46           | 1.86          |
| <i>Accuracy by Reasoning Category (<math>N = 40</math> per slice):</i> |                  |             |                  |               |
| Greater / Smaller / Equal                                              | 0.0%             | 100.0%      | 2.5%             | 100.0%        |
| Numerical Comparisons                                                  | 2.5%             | 90.0%       | 10.0%            | 80.0%         |
| Direct Comparisons                                                     | 0.0%             | 75.0%       | 55.0%            | 85.0%         |
| 2-Hop Transitive Comparisons                                           | 2.5%             | 30.0%       | 45.0%            | 65.0%         |
| 3-Hop Transitive Comparisons                                           | 2.5%             | 30.0%       | 37.5%            | 80.0%         |
| 4-Hop Transitive Comparisons                                           | 0.0%             | 27.5%       | 35.0%            | 77.5%         |

Table 12: Pretrained vs. Finetuned performance on held-out synthetic reasoning benchmarks ( $N = 240$  test examples under zero train-test leakage). Finetuning produces massive gains in exact match, collapsing perplexity from  $> 300$  down to  $< 2.2$ .

When we fine-tuned both models, Model H saw a massive leap in exact matches—jumping from a sluggish baseline of 1.25% Model L, on the other hand, stole the show. It pulled ahead with a stellar 81.25

## 4.3 Qualitative Analysis and Error Taxonomy

Table 13 presents qualitative generation examples from held-out test prompts, contrasting pretrained outputs against finetuned reasoning trajectories.

| Test Instance                                                        | Text and Mechanistic Generation Analysis                                                                                                                                                                                         |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Case 1: Numerical Comparison (Model H — Telugu)</b>               |                                                                                                                                                                                                                                  |
| Prompt                                                               | ప్రశ్న: 45 మరియు 73 సంఖ్యలను పోల్చండి. ఏది పెద్దది? జవాబు:                                                                                                                                                                       |
| Gold Target                                                          | 73                                                                                                                                                                                                                               |
| Pretrained Output                                                    | ఒక పెద్ద ప్రశ్న . జవాబు : ఒక పెద్ద ప్రశ్న ,                                                                                                                                                                                      |
| Finetuned Output                                                     | 73 > 45 కాబట్టి సమాధానం 73 . (Correct)                                                                                                                                                                                           |
| <b>Case 2: 4-Hop Transitive Distance Ordering (Model L — Nepali)</b> |                                                                                                                                                                                                                                  |
| Prompt                                                               | प्रश्न: म्याग्दी को दूरी मनाङ भन्दा बढी हो। मनाङ को दूरी पर्सा भन्दा बढी हो। पर्सा को दूरी नुवाकोट भन्दा बढी हो। नुवाकोट को दूरी स्याङ्जा भन्दा बढी हो। म्याग्दी, मनाङ, पर्सा, नुवाकोट र स्याङ्जा मध्ये कुनको दूरी कम हो? उत्तर: |
| Gold Target                                                          | स्याङ्जा (Syangja)                                                                                                                                                                                                               |
| Pretrained Output                                                    | म्याग्दी र मुस्ता डको दूरी मनाङ भन्दा कम हो । मुस्ता ड बाट मुस्ता ड जाने सडक पनि...                                                                                                                                              |
| Finetuned Output                                                     | क्रम म्याग्दी > मनाङ > पर्सा > वीरगञ्ज । त्यसैले स्याङ् जा कम हो । उत्तर स्याङ् जा । (Correct)                                                                                                                                   |

Table 13: Qualitative reasoning generation samples on held-out test prompts for Model H (Telugu) and Model L (Nepali).

## 4.4 Post-Finetuning Attention Analysis and Head Specialization

To investigate how gradient updates alter internal representations during reasoning adaptation, we re-applied our attention extraction pipeline to the finetuned checkpoints across all 14 sample scales, revealing two core mechanistic phenomena:

1. **Late-Layer Entropy Compression:** While early layers remain stable ( $\Delta H < 0.003$ ), the final layers (Layers 4 and 5) show sharp entropy reductions. For instance, Model H’s Layer 5 drops by  $\Delta H = -0.2246$  (from 1.255 down to 1.030), and Model L drops by  $\Delta H = -0.1983$ . Rather than diffusing probability mass, late-layer heads become intensely focused.
2. **Expansion of Query-to-Premise Distance:** Concurrently, late-layer attention distance expands ( $\Delta D = +0.4812$  in Model H;  $\Delta D = +0.3854$  in Model L). When generating token  $T$ , attention heads project long-range focal beams back to premise entities ( $t \in [5, 25]$ ), directly implementing an in-context pointer mechanism.

As illustrated in Figure ?? and Figure ??, finetuning transforms Layer 5 Head 6 from a diffuse, noisy state into a dedicated **reasoning pointer head**. It suppresses background noise and allocates over 70% of its total attention weight strictly onto the comparative operator tokens (e.g., కంటే / भन्दा) and target entities in the premise.

## 4.5 Key Findings: Data, Tokenization, and Reasoning Dynamics

**Q: How did data scale and quality differ between the models?**

Model H leveraged a massive 311M-word IndicCorp and web archive mix, whereas Model L relied heavily on scraped news (over 80%). Manual collection (47.2% for Telugu, 20.3% for Nepali) proved crucial for stripping boilerplate and stabilizing loss.

**Q: What tokenizer and corpus factors shaped Model L?**

A vocabulary size of  $V = 7,500$  was the sweet spot for Model L (1.89 fertility). Devanagari’s cleaner structure, combined with NFC normalization, completely wiped out out-of-vocabulary dropouts.

**Q: Why did pretraining fluency not guarantee reasoning success?**

Though Model H won pretraining perplexity (41.94 vs. 53.31), Model L crushed downstream logic—scoring an 81.25% exact match against Model H’s 58.75% and dominating 4-hop chains (77.5% vs. 27.5%).

**Q: What evidence explains Model L’s reasoning superiority?**

Four key factors explain the performance gap:

1. **Attention Flexibility:** Model L’s fluid attention ( $\bar{H} = 1.82$ ) adapted better to graph logic than Model H’s rigid representations ( $\bar{H} = 1.255$ ).
2. **Pointer Spans:** Model L cleanly expanded its Layer 5 attention span to track multi-hop chains, avoiding Model H’s local loops.
3. **Embedding Traps:** Model H suffered from severe greedy decoding copy loops due to rigid attractor basins.
4. **True Compression Parity:** When measured via Bits-Per-Byte, the initial perplexity gap vanished (0.0152 vs. 0.0162), proving both models compressed their languages with near-identical efficiency.

## 4.6 Bonus Ablation Study: Removing Positional Embeddings

Removing positional embeddings from Model H causes a total structural collapse. Without position awareness, predictive uncertainty skyrockets—test perplexity surges by 157.3% (41.94 to 107.92) and greedy decoding triggers an alarming **81.24% repetition loop** because the model loses all sense of token recency.

Mechanistically, attention entropy jumps by 33% to 55% and Layer 0’s attention distance doubles from 5.65 to 11.42, completely wiping out local recency diagonals. This failure boils down to two core theoretical breakdowns:

1. **Agglutinative Syntax Collapse:** In Telugu, word order and case markers dictate semantic roles (e.g., రాముడు రావణుడిని చూశాడు [*Rama saw Ravana*] vs. రావణుడు రాముడిని చూశాడు [*Ravana saw Rama*]). Because unrotated attention treats identical tokens identically regardless of position, the model cannot distinguish agent from patient.
2. **Loss of Induction Heads:** Without positional data, attention heads cannot select token  $t - 1$  over  $t - 8$  based on proximity, completely shutting down induction circuits and algorithmic pattern copying.